

Organ Donation & Sharing Platform

Comprehensive Project Documentation

1. Introduction

The **Organ Donation & Sharing Platform** (LifeGift Network) is a modern web application designed to connect organ donors with patients in critical need. The project features a user-friendly interface for donor registration, an organ search portal, and a real-time matching dashboard.

This document explains the architecture, components, and tools used to build the platform, ensuring a clear understanding of its inner workings.

2. Core Technologies Used

The application is built using a modern JavaScript/React stack. Here are the primary tools used:

- **React.js**: The core library used to build the User Interface components.
 - **Vite**: The build tool used to bundle the application. It is much faster than traditional tools like Create React App.
 - **React Router**: Manages navigation between different pages without reloading the web browser (Single Page Application experience).
 - **Supabase**: An open-source Firebase alternative configured with a PostgreSQL database. It stores our relational data (Users, Donors, Organs, Matches) and listens for real-time updates.
 - **Firestore**: Primarily configured for future integrations such as Cloud Firestore or Authentication, showing an alternative backend set up for scalability.
 - **Lucide React**: Provides the SVG icons used throughout the website (e.g., Heart, Shield, etc.).
 - **Vanilla CSS**: Traditional CSS files used for styling, keeping the design customized without relying strictly on heavyweight UI frameworks.
-

3. Root Configuration Files

These files configure the environment and scripts needed to start our web application.

- **package.json** : This file contains all the project metadata. It lists the dependencies (like `react` , `supabase-js` , `lucide-react`) required for the project and specifies scripts like `npm run dev` to start the local Vite server.
 - **vite.config.js** : Configuration file for Vite. It specifies that we are using the React plugin to compile JSX syntax.
 - **index.html** : The single HTML page served to the browser. The entire React application is injected into a `div` element with the id `root` in this file.
-

4. Application Entry Points (`src/` folder)

The execution of the React application begins here.

- **main.jsx** : The true entry point of the app. It imports `React` , selects the HTML `root` `div`, and renders the `App` component into it using `createRoot` .
- **App.jsx** : The root component. It wraps the entire application within `<Router>` from `react-router-dom` . It defines the layout by permanently displaying the `<Navbar />` at the top, the `<Footer />` at the bottom, and swapping the middle `<main>` content based on the URL (using `<Routes>` and `<Route>`).

- **index.css & App.css** : Contain global styling variables (like primary colors, fonts) and common layout classes used everywhere.
 - **supabaseClient.js** : Initializes the Supabase database connection. It creates a client utilizing our `supabaseUrl` and `supabaseKey` . This client is imported anywhere we need to read from or write to the database.
 - **firebase.js** : Holds the Firebase initialization configuration (API keys and project IDs). Currently sets up a Firestore reference (`db`).
-

5. Reusable UI Components (`src/components/`)

Components are reusable pieces of code that render specific parts of the screen.

- **Navbar.jsx / Navbar.css** :
 - **What it does:** Renders the top navigation bar with links like "Home", "Become a Donor", "Find Organ".
 - **How it works:** Uses `useLocation` to determine which page is active to highlight the link. Contains a mobile menu state variable (`isMobileMenuOpen`) toggled via an `onClick` event to support responsive design on phones.
 - **Footer.jsx / Footer.css** :
 - **What it does:** Renders the bottom footer displaying copyright information, emergency contact details, and quick links.
-

6. Application Pages (`src/pages/`)

These act as the main views. When a URL changes, React Router shows the corresponding page.

A. Home (`Home.jsx / Home.css`)

- **Purpose:** The landing page that greets the user.
- **Functionality:** It consists of static HTML structural components highlighting platform statistics (e.g., "10k+ Lives Saved"), feature cards ("Smart matching", "Real-Time Tracking"), and strong calls-to-action (CTAs) directing users to either Register or Find an organ.

B. Donate (`Donate.jsx / Donate.css`)

- **Purpose:** Allows generous individuals to register themselves as donors.
- **Functionality:**
 - Uses the `useState` React hook to track form inputs (`fullName` , `bloodType` , `organType` , etc.).
 - **handleSubmit** : An async function triggered when the user submits the form. It uses the Supabase client to insert a new row in the `donors` table. If the user specifies an organ, it simultaneously auto-inserts a listing into the `organs` table for the public Find registry.
 - If the database isn't connected, it catches the error and still shows a "Success" UI for display/simulation purposes to prevent breaking the flow.

C. Find (`Find.jsx / Find.css`)

- **Purpose:** Allows patients and medical coordinators to search for available organs.
- **Functionality:**
 - Tracks search terms (`searchTerm`), organ filters (`organFilter`), and blood type filters (`bloodTypeFilter`) using `useState` .
 - **handleSearch** : An async function that fetches records from the `organs` table via Supabase. If the fetch fails (e.g., network error), the application robustly falls back to a predefined array of

`MOCK_RESULTS` .

- Renders a grid of cards passing the filter. Includes a "View Details" button which opens a modal (popup window) displaying extended organ details.

D. Dashboard (`Dashboard.jsx` / `Dashboard.css`)

- **Purpose:** A real-time logistics dashboard intended for coordinators to track organ matches.
 - **Functionality:**
 - Uses the `useEffect` React hook to fetch initial matches from the `matches` table in Supabase when the page loads.
 - **Real-time Subscriptions:** Demonstrates advanced capabilities by creating a Supabase channel (`schema-db-changes`). It listens for any changes in the database and automatically updates the state (`setMatches`), updating the UI immediately without requiring a page refresh.
 - Rendered as an interactive table with stats headers. Users can click on individual matches to open detailed logs in a modal.
-

7. Summary

This project represents a sophisticated use of modern Web Development paradigms. By coupling **React's** component-based architecture for the frontend with **Supabase/Firebase** for backend database integrations, the application ensures real-time responsiveness, modularity, and high-performance routing. Every file and folder maintains a single responsibility, strictly separating styling (CSS), logic/structure (JSX), and database configuration.